

Nandhini MS

nandhinisubramaniam770@gmail.com

DATE:26/06/2026

WEEK 2

Module 5 - Python Backend Frameworks – Django, Flask & Fast API

HANDS-ON 2

TASK 1

Step 11: Define Models

```
from django.db import models
```

Department Model

```
class Department(models.Model): name =  
models.CharField(max_length=100) head_of_dept =  
models.CharField(max_length=100) budget =  
models.DecimalField(max_digits=12, decimal_places=2)  
def __str__(self):  
    return self.name
```

Course Model

```
class Course(models.Model): name =  
models.CharField(max_length=100) code =  
models.CharField(max_length=20, unique=True) credits =
```

```
models.IntegerField() department = models.ForeignKey(
Department, on_delete=models.CASCADE )
```

```
def __str__(self):
    return self.name
```

Student Model

```
class Student(models.Model): first_name =
models.CharField(max_length=100) last_name =
models.CharField(max_length=100) email =
models.EmailField(unique=True) department =
models.ForeignKey( Department,
on_delete=models.CASCADE ) enrollment_year =
models.IntegerField()
```

```
def __str__(self):
    return self.first_name + " " + self.last_name
```

Enrollment Model

```
class Enrollment(models.Model): student =
models.ForeignKey( Student, on_delete=models.CASCADE
) course = models.ForeignKey( Course,
on_delete=models.CASCADE ) enrollment_date =
models.DateField() grade =
models.CharField(max_length=5, null=True, blank=True)
```

```
class Meta:
    unique_together = [['student', 'course']]

def __str__(self):
    return self.student.first_name + " - " + self.course.name
```

MY WORKFLOW

The screenshot displays a VS Code editor interface for a Django project named 'coursemanager'. The Explorer sidebar on the left shows the project structure, including files like __pycache__, __init__.py, asgi.py, settings.py, urls.py, wsgi.py, migrations, tests.py, admin.py, apps.py, models.py, and views.py. The Outline sidebar shows the current file's structure, and the Timeline sidebar shows recent file saves. The main editor window displays the models.py file, which defines three models: Department, Course, and Student. The Department model has fields for name, head_of_dept, and budget. The Course model has fields for name, code, credits, and a foreign key to Department. The Student model has fields for first_name, last_name, email, and a foreign key to Department. The bottom panel shows the ChatGPT sidebar and a terminal window with a command prompt.

```
1 class Department(models.Model):
2     name = models.CharField(max_length=100)
3     head_of_dept = models.CharField(max_length=100)
4     budget = models.DecimalField(max_digits=12, decimal_places=2)
5
6     def __str__(self):
7         return self.name
8
9
10 # Course Model
11 class Course(models.Model):
12     name = models.CharField(max_length=100)
13     code = models.CharField(max_length=20, unique=True)
14     credits = models.IntegerField()
15     department = models.ForeignKey(
16         Department,
17         on_delete=models.CASCADE
18     )
19
20     def __str__(self):
21         return self.name
22
23
24 # Student Model
25 class Student(models.Model):
26     first_name = models.CharField(max_length=100)
27     last_name = models.CharField(max_length=100)
28     email = models.EmailField(unique=True)
29     department = models.ForeignKey(
30         Department,
31         on_delete=models.CASCADE
32     )
33     enrollment_year = models.IntegerField()
34
35     def __str__(self):
36         return self.first_name + " " + self.last_name
37
```

ChatGPT sidebar:

- EXPLORER
- COURSEMANAGER
- coursemanager
- __pycache__
- __init__.py
- asgi.py
- settings.py
- urls.py
- wsgi.py
- migrations
- __init__.py
- admin.py
- apps.py
- models.py
- tests.py
- urls.py
- views.py
- db.sqlite3
- manage.py

Terminal:

```
PS C:\Users\HP\coursemanager> & 'C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'C:\Users\HP\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54694' '--' 'C:\Users\HP\coursemanager\courses\models.py'
```

TASK 2

Task 2: Django ORM Queries

Goal: Use the Django ORM shell to perform CRUD operations and complex queries

Step 16: python manage.py shell Example: from
courses.models import Department, Course, Student,
Enrollment from django.db.models import Count, F from
django.db import connection cs =
Department.objects.create(name="Computer Science",
head_of_dept="Dr. John", budget=500000) ece =
Department.objects.create(name="Electronics",
head_of_dept="Dr. Smith", budget=400000)
Course.objects.create(name="Python Programming",
code="CS101", credits=4, department=cs)
Course.objects.create(name="Database Systems",
code="CS102", credits=3, department=cs)
Course.objects.create(name="Digital Electronics",
code="EC101", credits=4, department=ece)
Course.objects.create(name="Microprocessors",
code="EC102", credits=3, department=ece)
Student.objects.create(first_name="Rahul",
last_name="Kumar", email="rahul@gmail.com",
department=cs, enrollment_year=2024)
Student.objects.create(first_name="Priya",
last_name="Sharma", email="priya@gmail.com",
department=cs, enrollment_year=2024)

```
Student.objects.create(first_name="Arun", last_name="K",  
email="arun@gmail.com", department=ece,  
enrollment_year=2024)
```

```
Student.objects.create(first_name="Sneha",  
last_name="R", email="sneha@gmail.com",  
department=ece, enrollment_year=2025)
```

```
Student.objects.create(first_name="Karthik",  
last_name="M", email="karthik@gmail.com",  
department=cs, enrollment_year=2025) Query:
```

```
Course.objects.filter(department__name="Computer  
Science") Count:
```

```
Department.objects.values('name').annotate(course_count  
=Count('course')) Select Related: students =
```

```
Student.objects.select_related('department') for s in  
students: print(s.first_name, s.department.name) Update:
```

```
Department.objects.update(budget=F('budget')*1.1) Task  
3: Django Admin Interface Commands: python manage.py  
createsuperuser Register all models in admin.py.
```

```
Customize CourseAdmin: list_display =
```

```
['name','code','credits','department'] search_fields =
```

```
['name','code'] list_filter = ['department'] Create sample  
records through /admin and verify duplicate enrollment is  
prevented
```

